Code Explanation

Customer Order Processing Code Explanation

The provided code is a PySpark application that processes customer order data. It reads customer and order data from CSV files, cleans and transforms the data, and then loads it into Delta tables. The code also implements Slowly Changing Dimension (SCD) Type 2 for the ordersummary table and aggregates customer spend data.

Overview
The code is structured into several functions, each responsible for a specific task, such as reading data, cleaning data, joining data, and updating Delta tables. The main function orchestrates the entire process.

Step 1: Read Customer Data
This step reads customer data from a CSV file into a PySpark DataFrame. The `read_customer_data` function takes a SparkSession and a file path as input and returns a DataFrame with the customer data.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

return spark.read.option("header", "true").schema(customer_schema).c
sv(path)
```

Step 2: Read Order Data
This step reads order data from a CSV file into a PySpark DataFrame. The `read_order_data` function is similar to `read_customer_data`, but it uses a different schema.

```
order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])

return spark.read.option("header", "true").schema(order_schema).csv(
path)
```

Step 3: Clean Data
This step removes null and duplicate records from the customer and order DataFrames. The `clean_data` function takes a DataFrame as input and returns a cleaned DataFrame.

```
df_no_nulls = df.dropna()
df_clean = df_no_nulls.dropDuplicates()
```

```
return df_clean
```

Step 4: Calculate Total Amount
This step adds a new column "TotalAmount" to the order DataFrame by multiplying "PricePerUnit" and "Qty". The `calculate_total_amount` function takes an order DataFrame as input and returns the updated DataFrame.

```
return order_df.withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
```

Step 5: Join Customer and Order Data
This step joins the customer and order DataFrames on the "CustId" column. The `join_customer_order` function takes both DataFrames as input and returns the joined DataFrame.

```
return customer_df.join(order_df, "CustId", "inner").select(
    "CustId", "Name", "EmailId", "Region", "OrderId", "ItemName",
    "PricePerUnit", "Qty", "Date", "TotalAmount"
)
```

Step 6: Create Ordersummary Table
This step creates the ordersummary Delta table if it does not exist. The `create_ordersummary_table` function takes a SparkSession, catalog, and schema as input and creates the table.

```
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.{schema}.ordersummary (
    CustId STRING,
    Name STRING,
    EmailId STRING,
    Region STRING,
    OrderId STRING,
    ItemName STRING,
    PricePerUnit DOUBLE,
    Qty INT,
    Date DATE,
    TotalAmount DOUBLE,
    IsActive BOOLEAN,
    StartDate TIMESTAMP,
    EndDate TIMESTAMP
)
USING DELTA
""")
```

Step 7: Update SCD Type 2 Table
This step updates the ordersummary Delta table using SCD Type 2. The `update_scd_type2_table` function takes a SparkSession, catalog, schema, and joined DataFrame as input and updates the table.

```
new_data = joined_df.withColumn("IsActive", lit(True))
                    .withColumn("StartDate", current_timestamp())
                    .withColumn("EndDate", lit(None).cast(TimestampT
ype()))

# Perform merge operation
delta_table.alias("target")
    .merge(
        new_data.alias("source"),
        condition
    )
    .whenMatched()
    .updateExpr({
        "IsActive": "false",
        "EndDate": "current_timestamp()"
    })
    .whenNotMatched()
    .insertAll()
    .execute()
```

Step 8: Create Customer Aggregate Spend Table
This step creates the customeraggregatespend Delta table if it does not exist. The `create_customer_aggregate_spend_table` function takes a SparkSession, catalog, and schema as input and creates the table.

```
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.{schema}.customeraggregatespend
 (
    Name STRING,
    TotalAmount DOUBLE,
    Date DATE
)
USING DELTA
""")
```

Step 9: Aggregate Customer Spend
This step aggregates the TotalAmount by Name and Date from the ordersummary table and writes the result to the customeraggregatespend table. The `aggregate_customer_spend` function takes a SparkSession, catalog, and schema as input and performs the aggregation.

```
ordersummary_df = spark.table(f"{catalog}.{schema}.ordersummary").fi
lter("IsActive = true")
aggregated_df = ordersummary_df.groupBy("Name", "Date")
                               .agg(sum_("TotalAmount").alias("TotalA
mount"))
aggregated_df.write.format("delta").mode("overwrite").saveAsTable(f"
{catalog}.{schema}.customeraggregatespend")
```

Step 10: Main Function
The main function orchestrates the entire process by calling the various functions in the correct order.

```
def main():
    # ...
    customer_df = read_customer_data(spark, customer_path)
    order_df = read_order_data(spark, order_path)
    # ...
    joined_df = join_customer_order(customer_df_clean, order_df_with
_total)
    update_scd_type2_table(spark, catalog, schema, joined_df)
    # ...
    aggregate_customer_spend(spark, catalog, schema)
```